

Manipulando Datos en SQL

Lenguaje DML y Transacciones

BASE DE DATOS

20 de abril de 2026

Comprender cómo manipular datos en SQL usando DML y gestionar transacciones de forma segura

Definición:

Permite modificar los datos en las tablas

- INSERT → Agregar datos
- UPDATE → Modificar datos
- DELETE → Eliminar datos

Características del DML

- Cambios temporales
- Solo visibles para el usuario
- Requieren confirmación

COMMIT y ROLLBACK

COMMIT:

- Guarda cambios permanentemente

ROLLBACK:

- Deshace cambios

- Mantiene consistencia
- Permite pruebas antes de guardar
- Agrupa operaciones

- Conjunto de operaciones
- Garantiza integridad
- Inicia con DML

- COMMIT
- ROLLBACK
- DDL (commit implícito)
- Falla del sistema

SAVEPOINT

- Marca puntos intermedios
- Permite rollback parcial
- No funciona con TRUNCATE

INSERT

- Agrega nuevas filas
- Requiere valores
- Respeta tipos de datos

Ejemplo INSERT

```
INSERT INTO departments  
VALUES (70, 'IT', 100, 1700);
```

- Implícito → omitir columna
- Explícito → NULL

- SYSDATE → fecha actual
- USER → usuario actual

INSERT con Subconsulta

- Inserta datos desde SELECT
- Automatiza carga

INSERT Múltiple

- Inserción en varias tablas
- Condicional e incondicional

Errores INSERT

- Clave duplicada
- Tipo de dato incorrecto
- Valor demasiado largo
- Clave foránea inválida

UPDATE

- Modifica datos existentes
- Puede afectar múltiples filas

Ejemplo UPDATE

```
UPDATE employees  
SET department_id = 110  
WHERE employee_id IN (103,104);
```

Cuidado con UPDATE

- Sin WHERE → modifica todo
- Puede usar subconsultas

- Violación de claves
- Tipos incompatibles
- Integridad referencial

DELETE

- Elimina filas
- Puede usar WHERE

Ejemplo DELETE

```
DELETE FROM departments  
WHERE department_name = 'Finance';
```

DELETE vs TRUNCATE

DELETE:

- Permite rollback
- Más lento

TRUNCATE:

- Más rápido
- No permite rollback
- Libera espacio

- Claves primarias
- Claves foráneas
- Unique

- INSERT desde SELECT
- UPDATE con valores dinámicos
- DELETE con condiciones complejas

- Siempre usar WHERE
- Probar antes de COMMIT
- Usar transacciones
- Validar datos

Conclusión

- DML permite manipular datos
- Transacciones aseguran consistencia
- Buen uso evita errores críticos

¿Preguntas?